Design Specification: PARD Programming Language

Erdinç Daşkın

June 3, 2026

Contents

1 Introduction & Philosophy

PARD is a programming language designed around game design and development. It leverages FORTRAN's fast math and C interoperability. The name PARD is an abbreviation of Punching Card, which is an homage to the early days of FORTRAN.

2 Lexical Grammar

2.1 Variables

2.1.1 Variables and their types

Variables, just like most other programming languages, are a significant part of PARD. There's 4 basic variable types:

- **int**: A standard integer type for counts, loop counters, and IDs.
- **float**: A floating-point number for positions, physics calculations, and deltatime.
- **bool**: A boolean value (`true` or `false`) for state flags.
- **string[N]**: A fixed-length, ultra-fast character array of size N.

As you can see, strings are not dynamically sized - this is not only for easy operation in fortran, but it also introduces a good performance boost.

2.1.2 Variable definition

You can easily define a variable in the following way:

```
int myInt = 10
float myFloat = 10.5
bool myBool = true
string[6] myString = 'Hello!'
```

2.1.3 Simple usage of variables

You can easily use a variable in multiple ways, here are some:

```
int x = 2
int y = 3
int z = x + y
float f = 2.5
float g = 0.5
float h = f / g
```

You cannot mix types: by this I mean you can't do the following:

```
int x = 2
float y = 10.5
float z = x + y
```

This will raise an error, as not all types used together are of the same type.

2.1.4 Converting variables to other types

In the previous example, we had this:

```
int x = 2
float y = 10.5
float z = x + y
```

But what if we want to combine different types? Well, you can use simple parsing functions:
`float z = x + y.tofloat`
There exists tofloat, toint and tostring.

2.1.5 Array definition

Okay, how do we define arrays?
Before the type, put (n) where n is the lenght of said array.

```
(2)int myInt = [10, 5]
(3)float myFloat = [10.5, 3.14, 5.5]
(3)bool myBool = [true,false,false]
(2)string[6] myString = ['Hello ', 'World!']
```

2.1.6 Accessing values in an array

To get a part of an array, use an index. PARD is 1-index based, e.g. the first item is index 1, the second is index 2 etc.

```
(2)int myIntArray = [10, 5]
int myInt = myIntArray[2]
```

In this exaple, myInt will be 5.

2.1.7 Constants

One can easily define constants using the const keyword.

```
const (2)int myIntArray = [10, 5]
const float y = 10.5
```

2.2 Procedures

One may ask, what is the difference between procedures and functions in PARD. Well, the difference is, that functions always return a value, while not all procedures have to.
All procedure parameters are passed by reference by default.
There's 3 types of procedures:

2.2.1 Routines

Routines are procedures that don't return any value. They are defined by the rout keyword and require to have atleast 1 parameter.

```
rout myRoute(int x, int y){
x = y
}
```

Keep in mind, route parameters are by default passed by reference, and in this case, the original x passed will be modified.

2.2.2 Subroutines

Subroutines are routes that dont take any parameters. They are defined by the sub keyword.

```
sub mySub {
pass
}
```

the pass keyword simply does nothing.

2.2.3 Functions

Functions are like routes that return a value. They are defined by the keyword fn. They may or may not have parameters.

```
fn myFunc(int x, int y){
return x
}
```

2.2.4 Passing parameters by value

If you do not wish a routine/function to affect variables passed into it,
you may use the value indicator &:

```
rout myRoute(&int x, int y){
x = y
}
```

In this case, if a variable is passed as the parameter x, it wont change to y globally.

2.3 Engine procedures

Now, to actually be able to function correctly and have a drawing loop, there are 3 predefined procedures by PARD which must be used in all programs.

2.3.1 init

The init sub is called when the program starts and its meant to initialize variables/state.

```
sub init {
pass
}
```

The init sub should always be the first procedure in a file.

2.3.2 draw

The draw sub is called every game tick and is meant to draw stuff on the screen.

```
sub draw {
clear(WHITE)
}
```

By the way, the clear routine clears the screen to the passed color.

3 Compiler Architecture

There are 2 big components in PARD in the compiling proccess: the actual PARD transpiler, and a fortran compiler. The default compiler used by PARD is GFortran, tho you can use your own compiler by using the -k (keep) flag during compiling, which makes it so only the PARD transpiler is used and the result is the .f90 file

3.1 File extension

PARD files don't require any specific extension (like most other languages), however, the .pard extension is highly recomended (to allow IDE functionality like intellisense etc.)

3.2 The transpiling process

PARD is not really a compiler, as it transpiles to FORTRAN. It uses the raylib-c binding to have the game engine functionality (big thanks to Interkosmos btw) Because FORTRAN is not case-sensitive, so is PARD, as all keywords are converted to lowercase during transpiling. Because fortran doesnt allow for variable declaration mid-code, the transpiling proccess is 2pass:

3.2.1 1st pass

In the first pass, the transpiler scans the file for all variable declarations, and then puts them all at the start, and then continues to transpile.

3.2.2 2nd pass

In the second pass, the transpiler goes and transpiles the modified file into modern Fortran.